Octant v2 core
Security Review

Cantina Solo review by:
Zigtur, Lead Security Researcher

March 26, 2026

Contents

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

A security review is a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While the review endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that a security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Octant is an experiment in participatory public goods funding, utilizing Golem's native ERC-20 token, GLM. Developed by the Golem Foundation, Octant explores motivations for public goods support.

From Oct 14th to Oct 19th the security researchers conducted a review of octant-v2-core on commit hash d0a7b23f. A total of **7** issues were identified:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	1	1	0
Gas Optimizations	1	1	0
Informational	5	3	2
Total	**7**	**4**	**3**

2.1 Scope

The security review had the following components in scope for octant-v2-core on commit hash d0a7b23f:

```
src
├── constants.sol
├── errors.sol
├── core
│   ├── BaseStrategy.sol
│   ├── MultistrategyLockedVault.sol
│   ├── MultistrategyVault.sol
│   ├── PaymentSplitter.sol
│   ├── TokenizedStrategy.sol
│   └── libs
│       ├── DebtManagementLib.sol
│       └── ERC20SafeApproveLib.sol
├── factories
│   ├── BaseStrategyFactory.sol
│   ├── LidoStrategyFactory.sol
│   ├── MorphoCompounderStrategyFactory.sol
│   ├── MultistrategyVaultFactory.sol
│   ├── PaymentSplitterFactory.sol
│   ├── RegenStakerFactory.sol
│   ├── SkyCompounderStrategyFactory.sol
│   ├── yieldDonating
│   │   └── YearnV3StrategyFactory.sol
│   └── yieldSkimming
│       └── RocketPoolStrategyFactory.sol
├── mechanisms
│   ├── AllocationMechanismFactory.sol
│   ├── BaseAllocationMechanism.sol
│   ├── TokenizedAllocationMechanism.sol
│   ├── mechanism
│   │   ├── OctantQFMechanism.sol
│   │   └── QuadraticVotingMechanism.sol
│   └── voting-strategy
│       ├── ProperQF.sol
│       └── README.md
├── regen
│   ├── RegenEarningPowerCalculator.sol
```

```
│   ├── RegenStaker.sol
│   ├── RegenStakerBase.sol
│   └── RegenStakerWithoutDelegateSurrogateVotes.sol
├── strategies
│   ├── periphery
│   │   ├── BaseHealthCheck.sol
│   │   ├── BaseYieldSkimmingHealthCheck.sol
│   │   └── UniswapV3Swapper.sol
│   ├── yieldDonating
│   │   ├── MorphoCompounderStrategy.sol
│   │   ├── SkyCompounderStrategy.sol
│   │   ├── YearnV3Strategy.sol
│   │   └── YieldDonatingTokenizedStrategy.sol
│   └── yieldSkimming
│       ├── BaseYieldSkimmingStrategy.sol
│       ├── IYieldSkimmingStrategy.sol
│       ├── LidoStrategy.sol
│       ├── RocketPoolStrategy.sol
│       └── YieldSkimmingTokenizedStrategy.sol
└── zodiac-core
    ├── BaseStrategy.sol
    ├── DragonRouter.sol
    ├── LinearAllowanceExecutor.sol
    ├── ModuleProxyFactory.sol
    ├── SplitChecker.sol
    ├── TokenizedStrategy.sol
    ├── guards
    │   └── antiLoopHoleGuard.sol
    ├── modules
    │   ├── LinearAllowanceSingletonForGnosisSafe.sol
    │   ├── MethYieldStrategy.sol
    │   ├── OctantRewardsSafe.sol
    │   └── YearnPolygonUsdcStrategy.sol
    └── vaults
        ├── DragonBaseStrategy.sol
        ├── DragonTokenizedStrategy.sol
        ├── Passport.sol
        ├── TokenizedStrategy.sol
        └── YieldBearingDragonTokenizedStrategy.sol
```

3 Findings

3.1 Low Risk

3.1.1 Profit limit ratio may be reached after withdrawals

Severity: Low Risk

Context: BaseHealthCheck.sol#L166-L170

Description: The profit calculated in `BaseHealthCheck` is checked to be within a range of 0% to a configured profit limit ratio (up to 655%). This profit limit ratio check can fail when a user withdraws a substantial amount before a `report()` call, making the profit greater than the configured limit ratio.

Proof of Concept: Apply the following patch to import the proof of concept. It shows that `report()` reverts after user withdrawal.

```
  diff --git a/test/integration/strategies/YieldDonating/MorphoCompounderStrategy.t.sol
   ↪  b/test/integration/strategies/YieldDonating/MorphoCompounderStrategy.t.sol
  index a954a13..a79a9d1 100644
-  -- a/test/integration/strategies/YieldDonating/MorphoCompounderStrategy.t.sol
+  ++ b/test/integration/strategies/YieldDonating/MorphoCompounderStrategy.t.sol
  @@ -2,6 +2,7 @@
   pragma solidity ^0.8.25;

   import { Test } from "forge-std/Test.sol";
+  import "forge-std/console2.sol";
   import { MockERC20 } from "test/mocks/MockERC20.sol";
   import { MorphoCompounderStrategy } from
    ↪  "src/strategies/yieldDonating/MorphoCompounderStrategy.sol";
   import { BaseHealthCheck } from "src/strategies/periphery/BaseHealthCheck.sol";
  @@ -279,6 +280,55 @@ contract MorphoCompounderDonatingStrategyTest is Test {
           assertEq(limit, morphoLimit, "Available deposit limit should match Morpho
            ↪  vault limit when no idle assets");
       }

+      /// @notice Test available deposit limit with idle assets (TRST-M-8 fix)
+      function testWithdrawBeforeReportMakesItImpossible() public {
+          uint256 depositAmount = 1000000e6; // 1,000,000 USDC
+
+          // Airdrop idle assets to strategy to simulate undeployed funds
+          airdrop(ERC20(USDC), address(user), depositAmount);
+
+
+          // Initial balances
+          uint256 initialUserBalance = ERC20(USDC).balanceOf(user);
+          //uint256 initialStrategyAssets = IERC4626(address(strategy)).totalAssets();
+
+          // Deposit assets
+          vm.startPrank(user);
+          uint256 sharesReceived = IERC4626(address(strategy)).deposit(depositAmount,
↪  user);
+          vm.stopPrank();
+
+          // Verify balances after deposit
+          assertEq(ERC20(USDC).balanceOf(user), initialUserBalance - depositAmount,
↪  "User balance not reduced correctly");
+
+          console2.log("Shares received:", sharesReceived);
+
+          // 50K USDC profit is made
+          uint256 profitAmount = 50000e6; // 50,000 USDC profit
+          uint256 balanceOfMorphoVault =
↪  IERC4626(MORPHO_VAULT).balanceOf(address(strategy));
+          vm.mockCall(
+              address(IERC4626(MORPHO_VAULT)),
```

```
+             abi.encodeWithSelector(IERC4626.convertToAssets.selector,
↪  balanceOfMorphoVault),
+             abi.encode(depositAmount + profitAmount)
+         );
+
+
+         // @audit user withdraws before report is made
+         vm.startPrank(user);
+
↪  IERC4626(address(strategy)).redeem(IERC4626(address(strategy)).maxRedeem(user), user,
↪  user);
+         vm.stopPrank();
+
+         // Call report to harvest and donate yield, should revert
+         vm.startPrank(keeper);
+         (uint256 profit, uint256 loss) = IMockStrategy(address(strategy)).report();
+         vm.stopPrank();
+
+         console2.log("Profit:", profit);
+         console2.log("Loss:", loss);
+
+         vm.clearMockedCalls();
+     }
+
+
      /// @notice Test available deposit limit with idle assets (TRST-M-8 fix)
      function testAvailableDepositLimitWithIdleAssets() public {
          uint256 idleAmount = 1000e6; // 1,000 USDC idle assets
```

Recommendation: In such case, management can take action to unlock reports by calling `setDoHealthCheck` and avoid the check on the next report. This should be monitored so that the management is alerted and can take action.

Octant: Acknowledged.

Zigtur: Acknowledged.

3.2 Gas Optimization

3.2.1 Subtractions can be unchecked

Severity: Gas Optimization

Context: YieldSkimmingTokenizedStrategy.sol#L606-L607

Description: `_rebalanceDebtOnDragonTransfer` function executes `require` checks before subtraction.

```
function _rebalanceDebtOnDragonTransfer(address from, address to, uint256
↪  transferAmount) internal {
    YieldSkimmingStorage storage YS = _strategyYieldSkimmingStorage();
    StrategyData storage S = _strategyStorage();

    // Direct transfer: shares represent ETH value 1:1 in this system
    if (from == S.dragonRouter) {
        // Dragon sends shares: dragon loses debt obligation, users gain debt obligation
        require(YS.dragonRouterDebtInAssetValue >= transferAmount, "Insufficient dragon
        ↪  debt");
        YS.dragonRouterDebtInAssetValue -= transferAmount; // @audit could be unchecked
        YS.totalUserDebtInAssetValue += transferAmount;
    } else if (to == S.dragonRouter) {
        // User sends shares to dragon: users lose debt obligation, dragon gains debt
        ↪  obligation
        require(YS.totalUserDebtInAssetValue >= transferAmount, "Insufficient user
        ↪  debt");
        YS.totalUserDebtInAssetValue -= transferAmount; // @audit could be unchecked
        YS.dragonRouterDebtInAssetValue += transferAmount;
```

```
    }
}
```

Recommendation: The subtractions can be safely unchecked to save gas.

```
  diff --git a/src/strategies/yieldSkimming/YieldSkimmingTokenizedStrategy.sol
   ↪  b/src/strategies/yieldSkimming/YieldSkimmingTokenizedStrategy.sol
  index 2166673..99cb036 100644
-  -- a/src/strategies/yieldSkimming/YieldSkimmingTokenizedStrategy.sol
+  ++ b/src/strategies/yieldSkimming/YieldSkimmingTokenizedStrategy.sol
  @@ -604,12 +604,12 @@ contract YieldSkimmingTokenizedStrategy is TokenizedStrategy {
           if (from == S.dragonRouter) {
               // Dragon sends shares: dragon loses debt obligation, users gain debt
               ↪  obligation
               require(YS.dragonRouterDebtInAssetValue >= transferAmount, "Insufficient
               ↪  dragon debt");
-              YS.dragonRouterDebtInAssetValue -= transferAmount;
+              unchecked { YS.dragonRouterDebtInAssetValue -= transferAmount; }
               YS.totalUserDebtInAssetValue += transferAmount;
           } else if (to == S.dragonRouter) {
               // User sends shares to dragon: users lose debt obligation, dragon gains
               ↪  debt obligation
               require(YS.totalUserDebtInAssetValue >= transferAmount, "Insufficient user
               ↪  debt");
-              YS.totalUserDebtInAssetValue -= transferAmount;
+              unchecked { YS.totalUserDebtInAssetValue -= transferAmount; }
               YS.dragonRouterDebtInAssetValue += transferAmount;
           }
       }
```

Octant: Fixed in PR 318 commit d56a7a1.

Zigtur: Fixed. Subtraction have been unchecked, it is safe thanks to the `require` statement.

3.3 Informational

3.3.1 `SafeERC20` is imported twice in `MorphoCompounderStrategy`

Severity: Informational

Context: MorphoCompounderStrategy.sol#L4-L8

Description: `SafeERC20` library is imported twice in `MorphoCompounderStrategy`.

```
import { BaseHealthCheck } from "src/strategies/periphery/BaseHealthCheck.sol";
import { SafeERC20 } from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import { IERC20 } from "@openzeppelin/contracts/interfaces/IERC20.sol";
import { ITokenizedStrategy } from "src/core/interfaces/ITokenizedStrategy.sol";
import { SafeERC20 } from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
```

Recommendation: Remove one of the imports.

```
  diff --git a/src/strategies/yieldDonating/MorphoCompounderStrategy.sol
   ↪  b/src/strategies/yieldDonating/MorphoCompounderStrategy.sol
  index c0457c1..0a2ef48 100644
-  -- a/src/strategies/yieldDonating/MorphoCompounderStrategy.sol
+  ++ b/src/strategies/yieldDonating/MorphoCompounderStrategy.sol
  @@ -5,7 +5,6 @@ import { BaseHealthCheck } from
   ↪  "src/strategies/periphery/BaseHealthCheck.sol";
   import { SafeERC20 } from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
   import { IERC20 } from "@openzeppelin/contracts/interfaces/IERC20.sol";
   import { ITokenizedStrategy } from "src/core/interfaces/ITokenizedStrategy.sol";
-  import { SafeERC20 } from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

   /// @title MorphoCompounderStrategy
```

```
/// @author [Golem Foundation](https://golem.foundation)
```

Octant: Fixed in commit 4e87167.

Zigtur: Fixed. Import has been removed.

3.3.2 `totalUserDebtInAssetValue` variable naming

Severity: Informational

Context: YieldSkimmingTokenizedStrategy.sol#L33-L34

Description: The `totalUserDebtInAssetValue` variable name can let the reader think that it corresponds to the debt of the users. However, it corresponds to the value that the contract owes to the users and not the opposite.

Recommendation: Variable could be renamed. A name like `totalDebtOwedToUserInAssetValue` could be used.

Octant: Fixed in PR 318 commit b8c40de.

Zigtur: Fixed. `totalDebtOwedToUserInAssetValue` name is now used.

3.3.3 `_swapTo` internal function is not used

Severity: Informational

Context: UniswapV3Swapper.sol#L108-L171

Description: The `UniswapV3Swapper._swapTo` function is not used in any of the contract. Only the `_swapFrom` function is required.

Recommendation: As this function is not used, it can be safely removed.

Octant: Acknowledged. We will not fix as we may need it on other strategies.

Zigtur: Acknowledged.

3.3.4 Misleading comment in `YieldSkimmingTokenizedStrategy.withdraw`

Severity: Informational

Context: YieldSkimmingTokenizedStrategy.sol#L230

Description: The `YieldSkimmingTokenizedStrategy.withdraw` has a comment indicating `1 share = 1 ETH value`. While this is true in most cases, it can be false when a loss wasn't fully covered by the DragonRouter.

Recommendation: Consider modifying the comment to something like `1 share = 1 ETH value, expect in case of uncovered loss`.

Octant: Fixed in PR 318 commit 1797fc5.

Zigtur: Fixed. Comment is now more precise with this mention: `expect in case of uncovered loss`.

3.3.5 Keepers may not have incentives to call `bumpEarningPower`

Severity: Informational

Context: RegenStakerBase.sol#L995-L1000

Description: The fix in PR 265 aims to fix the competition finding "User Can Indefinitely Block Earning Power Reduction, Stealing Rewards from Other Stakers".

This PR modifies the `bumpEarningPower` function such that when there are not enough rewards for the keeper, the earning power still gets updated. However in such conditions, the keeper may not have any incentive to make the call.

There can be an incentive issue for a keeper to update the earning power when there is a blocklist/allowlist update that affects a `deposit`. There are 3 possible scenarios:

- The `deposit` has rewards: the keeper has an economic incentive to make the update (no issue).
- The `deposit` does not have rewards (or not enough), keeper has no rewards incentive:
  - The keeper is an active participant in the protocol: keeper has an incentive to make the update to reduce the total earning power and avoid dilution (no issue).
  - the keeper is not an active participant: here, there is no incentive (issue).

Recommendation: The keeper must be a participant in the protocol to have an incentive to pay for gas without earning rewards. This avoids the third scenario where the issue occurs.

Octant: Acknowledged.

Zigtur: Acknowledged.